AI Student Support Assistant

Muthalvan Project – Complete Starter Project

1. Project Overview

This project builds an AI Student Support Assistant that answers college-related questions using regulations, syllabus, FAQs and notices. The system uses Retrieval-Augmented Generation (RAG), tools and memory to provide useful, context-aware answers.

2. Architecture

Student → Streamlit UI → FastAPI Backend → RAG Retriever → Local Knowledge Base → Answer Generator. The prototype uses a lightweight local retrieval approach so it can be demonstrated without requiring a paid API.

3. Folder Structure

```
AI-Student-Support-Assistant/
■■■ backend/
■   ■■■ app.py
■■■ documents/
■   ■■■ college_info.txt
■■■ frontend/
■   ■■■ app.py
■■■ data/
■■■ requirements.txt
```

4. requirements.txt

```
fastapi
uvicorn
streamlit
requests
scikit-learn
```

5. Backend Code – backend/app.py

```
from fastapi import FastAPI
from pydantic import BaseModel
from pathlib import Path
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

app = FastAPI(title="AI Student Support Assistant")

DOC = Path(__file__).resolve().parent.parent / "documents" / "college_info.txt"
text = DOC.read_text(encoding="utf-8")
chunks = [x.strip() for x in text.split("\n\n") if x.strip()]

vectorizer = TfidfVectorizer(stop_words="english")
matrix = vectorizer.fit_transform(chunks)

memory = {}

class Question(BaseModel):
    question: str
    student_id: str = "demo_student"

def retrieve(question, k=3):
    q = vectorizer.transform([question])
    scores = cosine_similarity(q, matrix).flatten()
```

```
    ids = scores.argsort()[::-1][:k]
    return [chunks[i] for i in ids if scores[i] > 0]

@app.get("/")
def root():
    return {"status": "running", "project": "AI Student Support Assistant"}

@app.post("/ask")
def ask(item: Question):
    results = retrieve(item.question)
    if not results:
        answer = ("I could not find this information in the college knowledge base. "
                  "Please check the official college notice or contact the department.")
    else:
        answer = "Based on the college knowledge base:\n\n" + "\n\n".join(results)

    memory[item.student_id] = {
        "last_question": item.question,
        "last_answer": answer
    }
    return {"answer": answer, "sources": len(results)}
```

6. Frontend Code – frontend/app.py

```
import streamlit as st
import requests

st.set_page_config(page_title="AI Student Support Assistant")
st.title("■ AI Student Support Assistant")
st.write("Ask questions about regulations, syllabus, FAQs and notices.")

question = st.text_input("Enter your question")
student_id = st.text_input("Student ID", value="demo_student")

if st.button("Ask Assistant") and question:
    try:
        response = requests.post(
            "http://127.0.0.1:8000/ask",
            json={"question": question, "student_id": student_id},
            timeout=20
        )
        data = response.json()
        st.subheader("Answer")
        st.write(data["answer"])
        st.caption(f"Retrieved sources: {data['sources']}")
    except Exception:
        st.error("Backend is not running. Start FastAPI first.")
```

7. Sample Knowledge Base – documents/college_info.txt

```
Attendance:
Students should maintain the minimum attendance requirement specified by their college regulations.
If attendance is below the permitted level, the student should consult the department regarding eligibility.

Examinations:
Students must follow the examination timetable and regulations published by the institution.
Students should carry the required identity card and examination materials permitted by the rules.

Syllabus:
The syllabus contains course outcomes, units, topics, references and assessment information.
Students should use the latest syllabus issued by their department.

Leave:
Students should follow the institution's leave procedure and submit required approval or documentation
according to the applicable college rules.

Notices:
Important academic announcements, timetable changes and deadlines should be checked in the latest official notice.

FAQs:
For questions not answered by the knowledge base, students should contact the relevant department or office.
```

8. How to Run

```
1. Open Command Prompt / Terminal in the project folder.
2. Install packages:
   pip install -r requirements.txt

3. Start backend:
   uvicorn backend.app:app --reload

4. Open another terminal.
5. Start frontend:
   streamlit run frontend/app.py

6. Open the Streamlit address shown in the terminal.
7. Ask questions such as:
   - What is the attendance rule?
   - What does the syllabus contain?
   - Where should I check notices?
```

9. Key Agent Capabilities

RAG: Retrieves relevant college information before answering.

Tools: The FastAPI endpoint acts as a tool interface and can later be extended with timetable, notice and course lookup tools.

Memory: A simple student-level memory stores the latest question and answer. For a production system, this should be replaced with a persistent database.

10. Project Demonstration Flow

During the demo, show the folder structure, start the FastAPI server, start the Streamlit interface, ask two or three college-related questions, and explain how the system retrieves information from the knowledge base. Use only official/sample college information and clearly label sample data as sample data.

11. Future Enhancements

PDF/DOCX ingestion, semantic embeddings, a production vector database, authenticated student profiles, persistent memory, timetable tools, notice-date filtering, citations to source documents, multilingual support and a stronger LLM-based response layer can be added in the next version.